

What is Regression in Machine Learning?

Regression is used when the **target variable is continuous**. You apply regression models when your goal is to **predict quantities** like: House price, Income level, Temperature

(Q) continuous—meaning it can take on any real value within a range (including decimals) [A **range** refers to the span of possible values a variable can take. The **range** might be **bounded** (e.g., height cannot be negative).]

Common Regression Algorithms:

- **Simple Linear Regression:** Predicts using one independent variable.
Graph Shape: Straight Line
- **Multiple Linear Regression:** Uses multiple independent variables for prediction.
Graph Shape: Plane or hyperplane (multi-dimensional line)
- **Polynomial Regression:** Models nonlinear relationships by fitting a curve to the data.
Graph Shape: Curve (e.g., parabola)

(Q)Comparison Table:

Feature	Simple Linear Regression	Multiple Linear Regression	Polynomial Regression
Number of Independent Variables	1	2 or more	1 or more
Type of Relationship	Linear	Linear	Non-linear

Regressions

Simple
Linear
Regression

$$y = b_0 + b_1x_1$$

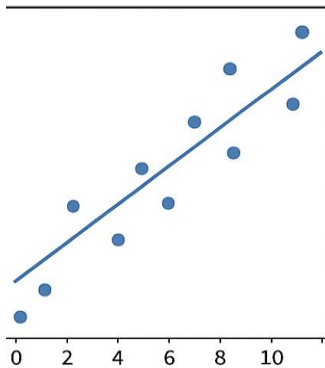
Multiple
Linear
Regression

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

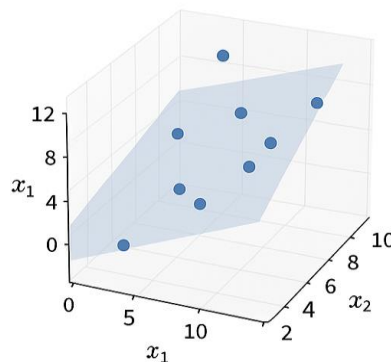
Polynomial
Linear
Regression

$$y = b_0 + b_1x_1 + b_2x_1^2 + \dots + b_nx_1^n$$

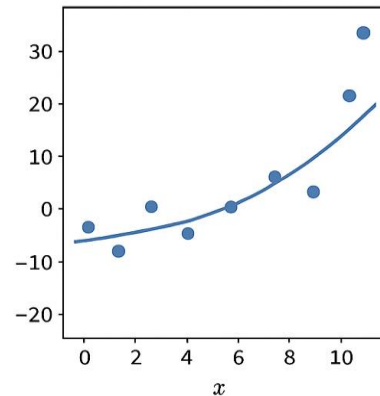
Simple Linear
Regression



Multiple Linear
Regression



Polynomial
Regression



(Q) An **independent variable** (also called a **feature**, **input**, or **predictor**) is the variable you use to **predict** the outcome (the **dependent** or **target** variable).

What is Classification in Machine Learning?

Classification is used when you want to categorize data into different classes or groups.

For example: Is an email spam or not? Is the patient healthy or sick?

Common Classification Algorithms:

- **Logistic Regression**
- **Decision Tree**
- **Random Forest**
- **K-Nearest Neighbors (KNN)**

Classification vs Regression in Machine Learning

Classification and regression are the two primary tasks in **supervised learning**. The key distinction lies in the **type of output** they predict:

🔍 **Classification** → Predicts **discrete outcomes** (e.g., *yes/no, categories*)

🔍 **Regression** → Predicts **continuous values** (e.g., house price, temperature).

(Q_) Characteristics of Discrete Outcomes:

- Can't take decimal values (e.g., you can't have 2.5 children).
- Values come from a **finite or countable set**.
- Often used in **classification** or **counting problems**.

(Q_) **Unsupervised learning** is a type of machine learning where the algorithm is trained on data **without labeled outputs**.

That means the model only sees **input features**, not the correct answers, and it tries to **find hidden patterns or structures** in the data on its own.

Examples in Real Life

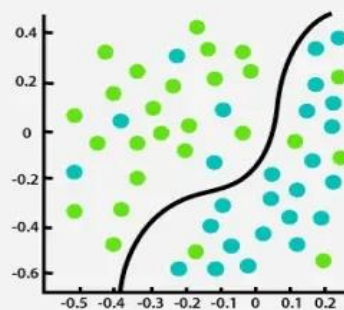
- Grouping similar news articles.

Detecting unusual patterns in credit card transactions (anomaly detection).

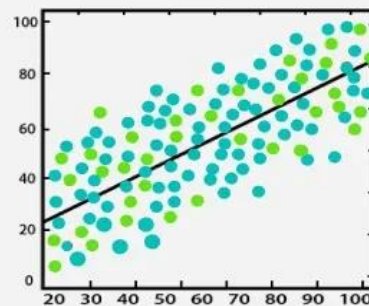
Both approaches **require labeled data for training but differ in their objectives**—

1)classification aims to find decision boundaries that separate classes

2)regression focuses on finding the best-fitting line to predict numerical outcomes



Classification



Regression

(Q_)Labeled data refers to a dataset where **each input (feature)** is paired with a known **output (label)**

Examples:

Features (Inputs)

Label (Output)

Age = 25, Income = 30K

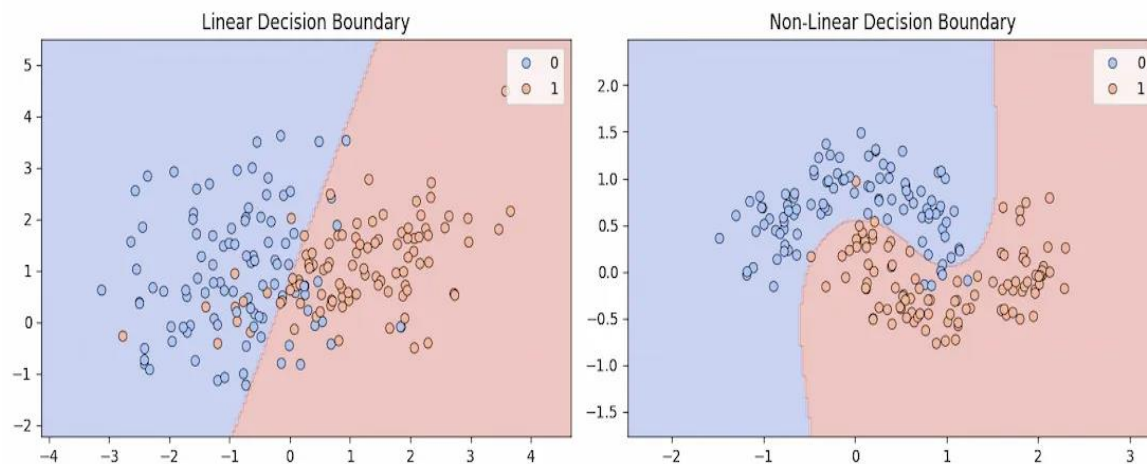
Buys Product = **Yes**

1. Decision Boundary in Classification

It is an **surface or line that separates data points into different classes in a feature space**. It can be **linear** (a straight line) or **non-linear** (a curve), depending on the complexity of the data and the algorithm used. For example:

A linear decision boundary might separate two classes in a 2D space with a straight line (e.g., logistic regression).

A **non-linear decision boundary** (e.g., from **Decision Trees**) can form **curved or irregular boundaries** to better fit more complex patterns in the data.



During training classifier **learns to partition the feature space by finding a boundary that minimizes classification errors**.

- For binary classification, this boundary separates data points into two groups (e.g., spam vs. non-spam emails).
- In multi-class classification, multiple boundaries are created to separate more than two classes

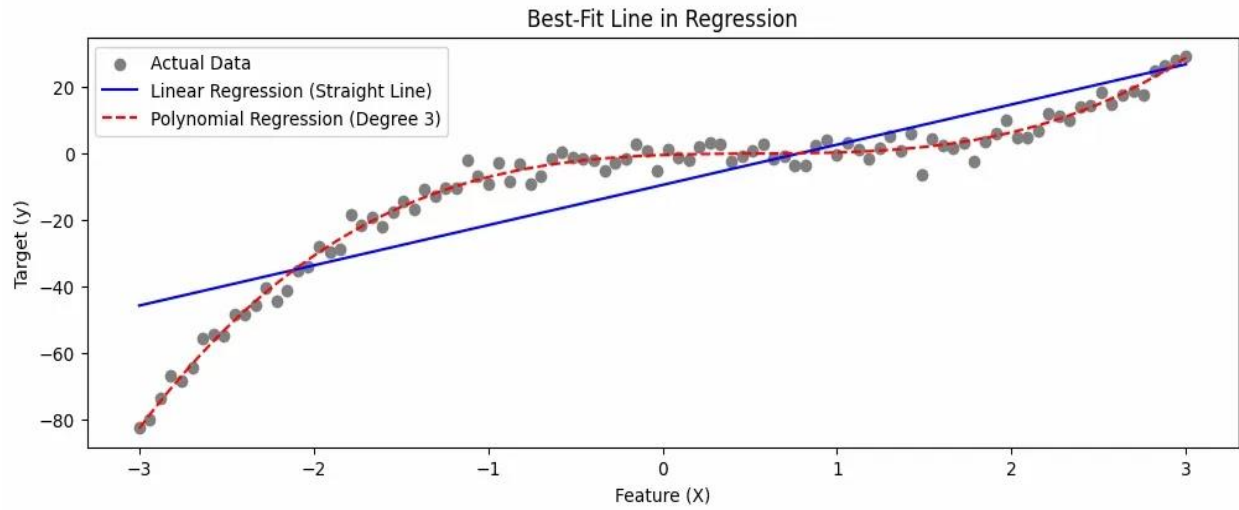
2. Best-Fit Line in Regression

In **regression**, a **best-fit line** (also called a **regression line**) models the relationship between **independent variables (inputs)** and a **dependent variable (output)**. It is used to **predict continuous numerical values** by capturing patterns and trends within the data.

This line helps the model **minimize the difference** between predicted and actual values, leading to more **accurate predictions**

The best-fit line **can be linear or non-linear**:

- A straight line is used for linear regression.
- Curves are used for more complex regressions, like polynomial regression



(Q_) Decision Boundary vs Best-Fit Line

Concept	Classification	Regression
Goal	Assign categories	Predict numerical values
Tool	Decision Boundary	Best-Fit Line

Comparison between Classification and Regression

Feature	Classification	Regression
Output type	In this problem statement, the target variables are discrete. Discrete categories (e.g., "spam" or "not spam")	Continuous numerical value (e.g., price, temperature).
Goal	To predict which category a data point belongs to.	To predict an exact numerical value based on input data.
Example problems	Email spam detection, image recognition, customer sentiment analysis.	House price prediction, stock market forecasting, sales prediction.
Evaluation metrics	Evaluation metrics like Precision, Recall, and F1-Score	Mean Squared Error, R2-Score, MAPE and RMSE.
Decision boundary	Clearly defined boundaries between different classes.	No distinct boundaries, focuses on finding the best fit line.
Common algorithms	Logistic regression, Decision trees, Support Vector Machines (SVM)	Linear Regression, Polynomial Regression, Decision Trees (with regression objective).

Linear Regression in Machine learning

Linear regression is a type of [supervised machine-learning algorithm](#) that learns from the labelled datasets and maps the data points with most optimized linear functions which can be used for prediction on new datasets. It assumes that there is a linear relationship between the input and output, meaning the output changes at a constant rate as the input changes. This relationship is represented by a straight line.

For example we want to predict a student's exam score based on how many hours they studied. We observe that as students study more hours, their scores go up. In the example of predicting exam scores based on hours studied. Here

- **Independent variable (input):** Hours studied because it's the factor we control or observe.
- **Dependent variable (output):** Exam score because it depends on how many hours were studied.

Why Linear Regression is Important?

- **Simplicity and Interpretability:** It's easy to understand and interpret, making it a starting point for learning about machine learning.

(Q_) [Interpretability in ML refers to how easily a human can understand:

- Why the model made a specific prediction
- How the model works internally
- What factors influenced the decision]

- **Predictive Ability:** Helps predict future outcomes based on past data, making it useful in various fields like finance, healthcare and marketing.
- **Basis for Other Models:** Many advanced algorithms, like logistic regression or neural networks, build on the concepts of linear regression.
- **Efficiency:** It's computationally efficient and works well for problems with a linear relationship.
- **Widely Used:** It's one of the most widely used techniques in both statistics and machine learning for regression tasks.

Best Fit Line in Linear Regression

In linear regression, the best-fit line is the straight line that most accurately represents the relationship between the independent variable (input) and the dependent variable (output). It is the line that minimizes the difference between the actual data points and the predicted values from the model.

1. Goal of the Best-Fit Line

The goal of linear regression is to find a straight line that minimizes the error (the difference) between the observed data points and the predicted values. This line helps us predict the dependent variable for new, unseen data.

2. Equation of the Best-Fit Line

For simple linear regression (with one independent variable), the best-fit line is represented by the equation

$$y = mx + b$$

Where:

- **y** is the predicted value (dependent variable)
- **x** is the input (independent variable)

- **m** is the slope of the line (how much y changes when x changes)(**Determine the effect of the feature** by setting m (slope)
- **b** is the intercept (the value of y when x = 0)(**What will be the starting value** by determining b (intercept))

The best-fit line will be the one that optimizes the values of m (slope) and b (intercept) so that the predicted y values are as close as possible to the actual data points.

So how do we get m and b?

1. Mathematical optimization (Analytical method)

- As I said before, it can be directly derived from the closed-form formula using the Normal Equation.
- This is absolutely correct considering the mean and covariance of the data.

2. Iterative optimization (Gradient Descent)

- Initially, m and b are initialized with a random value (such as 0 or a small value).
- It is then repeatedly updated with data until the cost function (such as MSE) is minimized.
- This process teaches the model what the best slope and intercept are.

In short

- **We do not manually fix m, b.**
- **The algorithm automatically finds the best value based on the data pattern.**
- So a machine learning model is an optimization process where model parameters (m, b) are taught.

When to use Mathematical Optimization (Normal Equation)?

- **If the dataset is small or medium in size.** (e.g. number of features n Not much, usually. $n < 10,000$ Okay until)
- Because the Normal Equation is a closed-form solution, directly inverting the matrix $X^T X$ Takes out:

$$\theta = (X^T X)^{-1} X^T y$$

- This method is simple, fast, and does not require artificial hyperparameters (learning rate, etc.).
 - More effective for small data.
-

When to use Gradient Descent?

- **If the dataset is too large or has too many features.**
 - Because in the Normal Equation $(X^T X)^{-1}$ Taking the inverse is computationally expensive and impossible in many places.
 - Gradient Descent iterative এবং scalable — step-by-step θ Update works.
 - In addition, Gradient Descent has many variants (Batch, Mini-batch, Stochastic) which are more commonly used in big data and online learning.
 - Hyperparameters (learning rate, iteration) need to be tuned.
-

Summary Table

Subject	Mathematical Optimization (Normal Equation)	Gradient Descent
Data size	Small or medium	Large or very large
Computational cost	More (Matrix inversion)	Less (iterative)
Hyperparameter	No need.	Need (learning rate, iterations)

Subject	Mathematical Optimization (Normal Equation)	Gradient Descent
Scalability	Less	More
Ease of implementation	Easy	A little complicated

Let me put it a little simpler:

- If the dataset is small → use **Normal Equation**
- Data is large or features are many → **Gradient Descent** is better

(Q) **How They Work Together**

- **Slope** controls the **tilt** of the line.
- **Bias** controls the **starting point** of the line.
- During training (via gradient descent), the algorithm adjusts m and b to **minimize the error** between predictions and actual values.

What is Best Fit Line?

- A straight line like we want $y = mx + c$
- $y = m_0 + m_1x$

Which will fit **the data points with as few errors as possible** .

- We call this mistake **an error** or **residual** :

$$e_i = y_i - \hat{y}_i$$

The basic idea of finding the Best Fit Line

We want to minimize **the errors** as much as possible.

To do this, we minimize **the Sum of Squared Errors (SSE)** :

$$SSE = \sum_{i=1}^n (and_i - (\theta_0 + i_1 x_i))^2$$

Step-by-Step Process

1 Creating a Cost Function

The cost function (MSE) will be:

$$J(\theta_0, i_1) = \frac{1}{n} \sum_{i=1}^n (and_i - (\theta_0 + i_1 x_i))^2$$

2 Finding θ_0 and θ_1

It can be found in two ways:

(a) Analytical Method (Normal Equation)

Formula:

$$i_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(and_i - a\bar{n}d)}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$i_0 = a\bar{n}d - i_1 \bar{x}$$

Here:

- $\bar{x}$ = Average of X
- $a\bar{n}\bar{d}$ = Average of Y

(b) Iterative Method (Gradient Descent)

1. Start with any value for θ_0 and θ_1 (random or 0)
2. Find the gradient:

$$\frac{\partial J}{\partial \theta_0} = -\frac{2}{n} \sum (a n d_i - a \hat{n} d_i)$$

$$\frac{\partial J}{\partial \theta_1} = -\frac{2}{n} \sum (a n d_i - a \hat{n} d_i) x_i$$

3. Update:

$$i_0 := i_0 - a \cdot \frac{\partial J}{\partial \theta_0}$$

$$i_1 := i_1 - a \cdot \frac{\partial J}{\partial \theta_1}$$

4. Repeat this process until the cost function is minimized.

3. Minimizing the Error: The Least Squares Method

To find the best-fit line, we use a method called [Least Squares](#). The idea behind this method is to minimize the sum of squared differences between the actual values (data points) and the predicted values from the line. These differences are called residuals.

The formula for residuals is:

$$\text{Residual} = y_i - \hat{y}_i$$

Where:

- y_i is the actual observed value
- $\hat{y}_i$ is the predicted value from the line for that x_i

The least squares method minimizes the sum of the squared residuals:

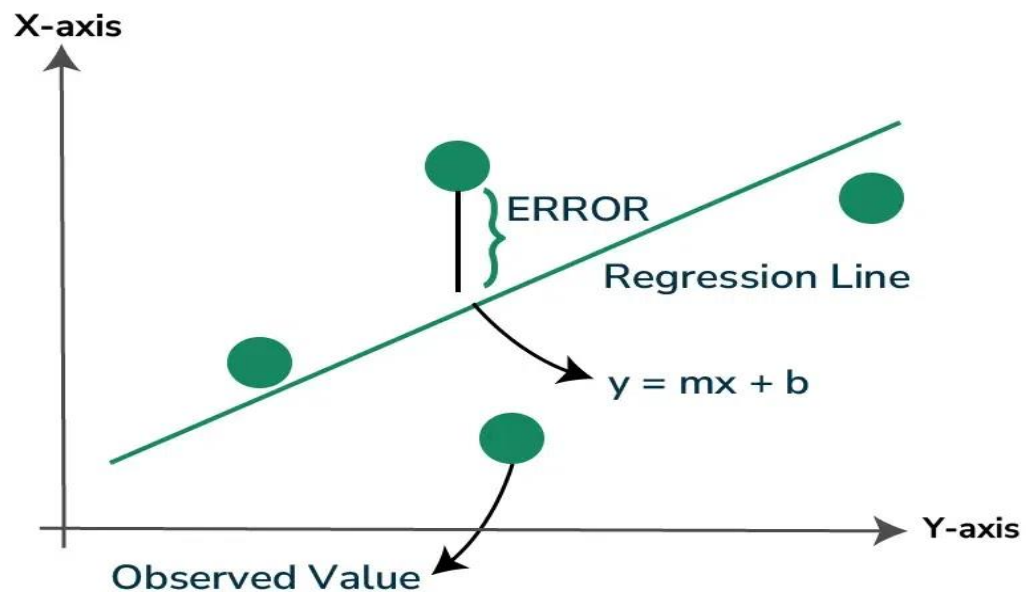
$$\text{Sum of squared errors (SSE)} = \sum (y_i - \hat{y}_i)^2$$

This method ensures that the line best represents the data where the sum of the squared differences between the predicted values and actual values is as small as possible.

Least Square Method

The Least Square method is a popular mathematical approach used in data fitting, regression analysis, and predictive modeling. It helps find the best-fit line or curve that minimizes the sum of squared differences between the observed data points and the predicted values

Least Square Method



Why use the Least Square Method?

we try to get an equation of a line that fits best to the given data points with the help of the Least Square Method.

Least Square Method Formula

The Least Square Method formula finds the [best-fitting line](#) through a set of data points. For a simple linear regression, which is a line of the form $y=mx+c$, where y is the dependent variable, x is the independent variable, a is the slope of the line, and b is the y-intercept, the formulas to calculate the slope (m) and intercept (c) of the line are derived from the following equations:

.

Steps to find the line of Best Fit by using the Least Squares Method :

- Step 1: Denote the independent variable values as x_i and the dependent ones as y_i .
- Step 2: Calculate the average values of x_i and y_i as X and Y .
- Step 3: Presume the equation of the line of best fit as $y = mx + c$, where m is the slope of the line and c represents the intercept of the line on the Y-axis.
- Step 4: The slope m can be calculated from the following formula:

$$m = [\Sigma (X - x_i) \times (Y - y_i)] / \Sigma (X - x_i)^2$$

- Step 5: The intercept c is calculated from the following formula:

$$c = Y - mX$$

Thus, we obtain the line of best fit as $y = mx + c$, where values of m and c can be calculated from the formulae defined above.

These formulas are used to calculate the parameters of the line that best fits the data according to the criterion of the least squares, minimizing the sum of the squared differences between the observed values and the values predicted by the linear model.

3. When should I use SSE?

- When we are training a model (using an iterative method like Gradient Descent), SSE is used as the loss .
 - Let's calculate SSE with each update to see if the error is decreasing.
 - The goal of our learning process is to minimize SSE.
-

4. Advantages and disadvantages of Normal Equation

- Normal Equation Directly θ_0, θ_1 outputs it all at once , so it doesn't require iteration like gradient descent.
 - However, the computational cost is higher for large or high-dimensional data.
 - Then, an attempt is made to reduce the SSE step by step using gradient descent.
-

Summary

- SSE is a loss function that tells how accurate the model's predictions are.
- Using the Normal Equation, we get the mathematical solution to minimize SSE.
- With Gradient Descent we reduce SSE step-by-step θ_0, θ_1 .

4. Interpretation of the Best-Fit Line

- **Slope (m):** The slope of the best-fit line indicates how much the dependent variable (y) changes with each unit change in the independent variable (x). For example if the slope is 5, it means that for every 1-unit increase in x , the value of y increases by 5 units.
- **Intercept (b):** The intercept represents the predicted value of y when $x = 0$. It's the point where the line crosses the y -axis.

In linear regression some hypothesis are made to ensure reliability of the model's results.

Limitations

- **Assumes Linearity:** *The method assumes the relationship between the variables is linear. If the relationship is non-linear, linear regression might not work well.*
- **Sensitivity to Outliers:** *Outliers can significantly affect the slope and intercept, skewing the best-fit line.*

(Q)when the actual relationship is non-linear (curved or complex), a straight line won't capture the pattern properly, leading to:

- High prediction errors
- Poor model performance
- Low R^2 score (coefficient of determination)

Meaning: Linear regression assumes that the relationship between **independent variables (X)** and the **dependent variable (y)** is **linear**.

- **Equation form:**

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- ☒ Works well if:
 - The data forms a straight-line pattern.
- ☐ Fails if:
 - The relationship is **non-linear** (e.g., exponential, quadratic, etc.)
 -

(Q)**outliers** — data points that are **far away from the rest of the data** — can pull the line toward themselves, causing: Linear regression uses **least squares error**, which **squares** the errors. So, large errors (from outliers) have a **huge impact**

- Incorrect slope (m)
- Incorrect intercept (c)
- Poor predictions for most data points

(Q) Example:

Suppose you're predicting student scores based on study time. Most students follow a pattern, but one student studied 2 hours and scored 100 (outlier).

This one point can **drag the line up**, even though it's not representative — leading to wrong predictions for the rest.

(Q) Definitions:

Term	Meaning
Linear Variable (Effect)	The change in the variable causes a proportional, constant change in the target. Fits a straight-line relationship.
Non-Linear Variable (Effect)	The change in the variable causes a non-proportional, varying change in the target. The effect is curved or complex .

(Q) Example:

1. Linear Variable Example:

Study Hours (x)	Score (y)
1	50
2	60
3	70

$$y=10x+40$$

✓ Here, **study hours** has a **linear effect** on the **score**.

(Q_2). Non-Linear Variable Example:

Medicine Dose (x) Health Improvement (y)

1	20
2	50
3	65
4	68

Here, as dose increases, improvement increases rapidly at first, then flattens out. This is a **non-linear effect** — could be modeled by a **logarithmic or sigmoid function**.

Hypothesis function in Linear Regression

In linear regression, the hypothesis function is the equation used to make predictions about the dependent variable based on the independent variables. It represents the relationship between the input features and the target output.

For a simple case with one independent variable, the hypothesis function is:

$$h(x)=\theta_0+\theta_1x$$

Where:

- $h(x)$ (or $\hat{y}$) is the predicted value of the dependent variable (y).
 - x is the independent variable.
 - θ_0 is the intercept, representing the value of y when x is 0.
 - θ_1 is the slope, indicating how much y changes for each unit change in x .
-
- **Example:**
 - Predicting a person's salary (y) based on their years of experience (x).

For **multiple linear regression** (with more than one independent variable), the hypothesis function expands to:

$$h(x_1, x_2, \dots, x_n) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

Where:

- $x_1, x_2, \dots, x_n$ are the independent variables.
- θ_0 is the intercept.
- $\theta_1, \theta_2, \dots, \theta_n$ are the coefficients, representing the influence of each respective independent variable on the predicted output.

(Q) Use Case of Multiple Linear Regression

Multiple linear regression allows us to analyze relationship between multiple independent variables and a single dependent variable. Here are some use cases:

- **Real Estate Pricing:** In real estate MLR is used to predict property prices based on multiple factors such as location, size, number of bedrooms, etc. This helps buyers and sellers understand market trends and set competitive prices.

Cost function for Linear Regression

The difference between the predicted value $\hat{Y}$ and the true value Y and it is called [cost function](#) or the [loss function](#).

In Linear Regression, the Mean Squared Error (MSE) cost function is employed, which calculates the average of the squared errors between the predicted values $\hat{y}_i$ and the actual values y_i . The purpose is to determine the optimal values for the intercept θ_1 and the coefficient of the input feature θ_2 providing the best-fit line for the given data points. The linear equation expressing this relationship is $\hat{y}_i = \theta_1 + \theta_2 x_i$

MSE function can be calculated as:

$$\text{Cost function}(J) = \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

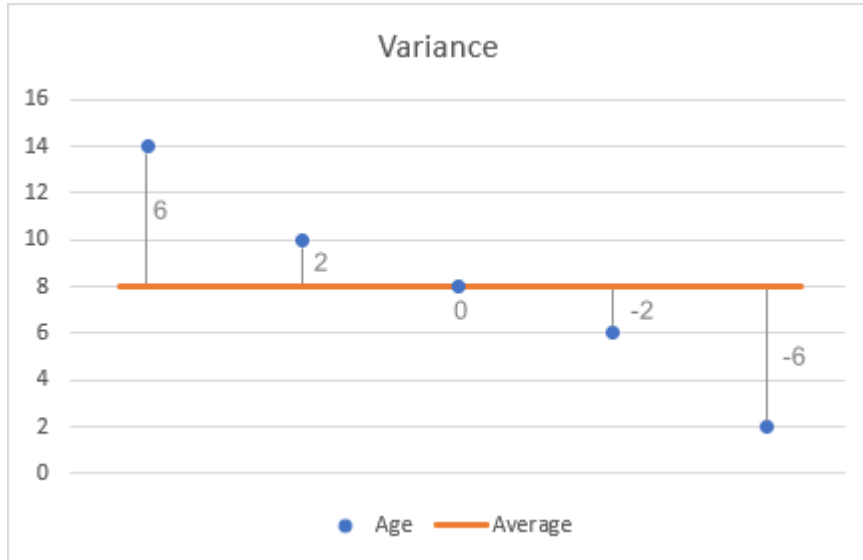
Utilizing the MSE function, the iterative process of gradient descent is applied to update the values of θ_1 & θ_2 . This ensures that the MSE value converges to the global minima, signifying the most accurate fit of the linear regression line to the dataset

This process involves continuously adjusting the parameters (θ_1) and (θ_2) based on the gradients calculated from the MSE. The final result is a linear regression line that minimizes the overall squared differences between the predicted and actual values, providing an optimal representation of the underlying relationship in the data.

Now we have calculated loss function we need to optimize model to mitigate this error and it is done through gradient descent.

❓ **Bias** = the model **directly underfits** .

❓ **Variance** = ModelOverfits to **small changes in the training data** .



Quick checklist

1. Train error $\approx$ Test error and both **are high** $\rightarrow$ **High Bias** (underfitting).
2. Train error **low**, Test error **high** $\rightarrow$ **High Variance** (overfitting).
3. Train low & Test low $\rightarrow$ good (balanced).

Evaluation Metrics for Linear Regression

A variety of [evaluation measures](#) can be used to determine the strength of any linear regression model. These assessment metrics often give an indication of how well the model is producing the observed outputs.

The most common measurements are:

1. Mean Square Error (MSE)

[Mean Squared Error \(MSE\)](#) is an evaluation metric that calculates the average of the squared differences between the actual and predicted values for all the data points. The difference is squared to ensure that negative and positive differences don't cancel each other out.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Here,

- n is the number of data points.
- y_i is the actual or observed value for the i th data point.
- $\hat{y}_i$ is the predicted value for the i th data point.

MSE is a way to quantify the accuracy of a model's predictions. MSE is sensitive to outliers as large errors contribute significantly to the overall score.

2. Mean Absolute Error (MAE)

[Mean Absolute Error](#) is an evaluation metric used to calculate the accuracy of a regression model. MAE measures the average absolute difference between the predicted values and actual values.

Mathematically MAE is expressed as:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Here,

- n is the number of observations
- y_i represents the actual values.
- $\hat{y}_i$ represents the predicted values

Lower MAE value indicates better model performance. It is not sensitive to the outliers as we consider absolute differences.

3. Root Mean Squared Error (RMSE)

It is a usually used metric in regression analysis and machine learning to measure the goodness of fit of a predictive model, especially when the predictions are continuous numerical values.

Mathematical Formula

The formula for RMSE for a data with 'n' data points is as follows:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - y_i)^2}$$

Where:

- RMSE is the Root Mean Squared Error.
- x_i represents the actual or observed value for the i-th data point.
- y_i represents the predicted value for the i-th data point.

Characteristic:

- 1) Same unit as target
- 2) High (sensitive to outliers)

4. (R-squared)

What percentage of the variability or variance was explained by our regression model in the actual target data. In general, the better the model matches the data, the greater the R-squared number.

In mathematical notation, it can be expressed as:

In regression, R^2 Score is often considered as an alternative to accuracy.

$$R^2 = 1 - \frac{\sum (and - a\hat{n}d)^2}{\sum (and - a\bar{n}d)^2}$$

Where,

- and = Actual values
- $a\hat{n}d$ = Predicted values
- $a\bar{n}d$ = Mean of actual values

That is,

- $R^2 = 1 \rightarrow$ Model perfectly fits the data (All variation explained) |
- $R^2 = 0 \rightarrow$ Model does not explain any variance, only the mean predicted will be equal.
- $R^2 < 0 \rightarrow$ Model worse than baseline mean

R^2 explains how well the model captures variability:

- $R^2 = 1 - (\text{Error of Model} / \text{Error of Baseline})$

When to Use

Model comparison

Measure how well model explains variance (How far apart are the data points from the mean)

5. Adjusted R-Squared Error

Adjusted R^2 is a version of R^2 that penalizes extra predictors. It helps you decide whether adding a variable actually improved the model (beyond just using more parameters)

$$\text{Adjusted } R^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

where

- R^2 = ordinary coefficient of determination,
- n = number of observations,
- p = number of predictors (independent variables)

Range: $(-\infty, 1]$ (It can be negative if the model is worse than predicting the mean.)

When to Use

- Multiple regression
- Model comparison
- Feature selection

Suppose:

We're predicting **house price (Y)** using:

- **Model 1:** Only one feature → house size (X_1)
- **Model 2:** Two features → house size (X_1) and number of rooms (X_2)

Step 2: Example Data

Model	R^2	n (data points)	k (predictors)
Model 1	0.80	20	1
Model 2	0.83	20	2

Step 3: Calculate Adjusted R^2

Model 1:

$$R_{adj}^2 = 1 - \left(\frac{(1 - 0.80)(20 - 1)}{20 - 1 - 1} \right) = 1 - \left(\frac{0.20 \times 19}{18} \right) = 1 - 0.211 = 0.789$$

Model 2:

$$R_{adj}^2 = 1 - \left(\frac{(1 - 0.83)(20 - 1)}{20 - 2 - 1} \right) = 1 - \left(\frac{0.17 \times 19}{17} \right) = 1 - 0.190 = 0.810$$

✓ **Step 4: Interpretation**

- **Model 1** → Adjusted $R^2 = 0.789$
- **Model 2** → Adjusted $R^2 = 0.810$

Since Adjusted R^2 increased, it means adding “number of rooms” improved the model.

Key Insights

1. R^2 always increases with more features → may mislead.
2. Adjusted R^2 penalizes unnecessary features.
3. Model selection:
 - If R^2 increases and Adjusted R^2 increases → feature is useful.
 - If R^2 increases but Adjusted R^2 decreases → feature remove.
4. Single variable regression → R^2 and Adjusted R^2 remain the same.

(Q_) MSE (Mean Squared Error): Example

Medical dosage prediction

- A large wrong dosage prediction could be dangerous, so large errors must be heavily penalized.

MAE (Mean Absolute Error): Example:

- Delivery time estimation
 - If you predict 30 minutes but the real time is 32 minutes, the error is 2 minutes.
 - A few rare large delays shouldn't skew the whole error metric.

When should I use which one?

1. Use MAE when:

- Your data has a high number of outliers or large errors .
- You want large errors to not have too much impact on the model's performance .
- Want to know the actual meaning of Error (e.g. money, kilograms), because MAE is in the original unit.
- Usually when you want to make the model robust .

2. Use MSE when:

- Your data has few outliers or you want to give more importance to large errors .
- You want to penalize the model to clean it of major errors.

- Smooth gradients are needed in training (such as in Gradient Descent) because the derivative of MSE is continuous and smooth.
- When you want to control the model's performance very carefully .

Example data

Let's say our actual data (y) and predicted data ($\hat{y}$) are like this:

Number Actual value (y) Predicted value ($\hat{y}$) Error ($y - \hat{y}$)

1	10	12	-2
2	8	9	-1
3	12	11	1
4	9	8	1
5	10	100 (Outlier)	-90

Step 1: MAE calculation

Formula:

$$MAE = (|-2| + |-1| + |1| + |1| + |-90|) / 5$$

$$MAE = (2 + 1 + 1 + 1 + 90) / 5 = (95 / 5) = 19$$

→ Outliers are large here, but there is a linear effect when calculating the average .

Step 2: MSE calculation

$$MSE = (-2)^2 + (-1)^2 + (1)^2 + (1)^2 + (-90)^2 / 5$$

$$MSE = (4 + 1 + 1 + 1 + 8100) / 5 = (8107 / 5) = 1621.4$$

Formula:

→ Here the error of the outlier (90) has been squared to 8100 , resulting in a much higher MSE.

Step 3: Comparison

- IS → 19
- MSE → 1621.4

Here we see that the MSE is very large due to the outliers, and the model performance looks very bad, even though the error was low at the remaining points.

The MAE minimizes the impact of the outliers, so it is more reflective of the original average performance of the data.

Why does MAE reduce the impact of outliers?

- MAE only takes the absolute value of the error , not the square.
- So 90 error and 9 error → both add proportionally .
- In MSE, 90 error → 8100 penalty is given, which dominates everything.

(Q)Regularization Techniques for Linear Models

What is Regularization?

Regularization helps to:

- Prevent overfitting
- Reduce model complexity
- Handle multicollinearity

1. Lasso Regression (L1 Regularization)

[Lasso Regression](#) is a technique used for regularizing a linear regression model, it adds a penalty term to the linear regression objective function to prevent [overfitting](#).

The objective function after applying lasso regression is:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^n |\theta_j|$$

-
- the first term is the least squares loss, representing the squared difference between predicted and actual values.
- the second term is the L1 regularization term, it penalizes the sum of absolute values of the regression coefficient θ_j . (absolute value of each weight)
-

2. Ridge Regression (L2 Regularization)

[Ridge regression](#) is a linear regression technique that adds a regularization term to the standard linear objective. Again, the goal is to prevent overfitting by penalizing large coefficient in linear regression equation. It useful when the dataset has multicollinearity where predictor variables are highly correlated.

The objective function after applying ridge regression is:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^n \theta_j^2$$

-
- the first term is the least squares loss, representing the squared difference between predicted and actual values.
- the second term is the L2 regularization term, it penalizes the sum of square of values of the regression coefficient θ_j .

Using the model to understand feature importance, the weight (coefficient) and loss function are changed .

Based on the model:

Our linear regression model

Based on the model:

Our linear regression model:

$$\hat{y} = \sum_{j=1}^p w_j x_j + b$$

- Every feature x_j 's weight w_j The model learns.
 - A larger weight means the feature has a greater impact on prediction.
 - A small or zero weight means the feature has little or no impact.
-

2. The role of regularization

L1 (Lasso):

- L1 regularization penalty $\lambda \sum |w_j|$ There is.
- optimization tries to minimize both loss and this penalty
- As a result, features whose retention means more penalty but less profit (loss is not reduced) have their weight shrunk to 0.
- That is, if the importance of the feature is low, its weight is 0.

L2 (Ridge):

- L2 penalty $\lambda \sum w_j^2$ There is.
- Features that are less important have their weight reduced, but not set to 0.

3. Mathematical Explanation: Sensitivity of Loss wrt Feature Weight

We can see, if any feature x_j its weight w_j If the model's loss function (error) decreases or increases significantly when it changes, then that is important.

Partial derivative of loss w.r.t w_j :

$$\frac{\partial J}{\partial w_j} = -\frac{2}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)}) x_j^{(i)} + \text{regularization term}$$

- If this gradient is large, in the weight update w_j Changing it has a big impact on the model's loss $\rightarrow$ the feature is important.
- If the gradient is close to zero, changing the weight has less impact on the model's loss $\rightarrow$ the feature is less important.

8. Practical process:

- The weights are optimized while training the model.
- The model considers features that have a large weight value to be necessary.
- A small or 0 weight value considers the feature unnecessary.

5. In short:

Formula for determining feature importance

Explanation

Big weight w_j

The feature contributes more to prediction

Small or 0 weight

The feature has little or no contribution.

Gradient magnitude (loss-sensitivity)

How much does weight loss change with weight change?

L1 (Lasso) এর Mathematical Property:

- The L1 penalty makes the optimization solution sparse — that is, many $\ln_j=0$ Yes.
- This works well when:

The number of features is large (p is very large), and we want to do feature selection.

- Of the many features, only a few are important, the rest are unnecessary.
 - Data-fitting loss requires sparsity in the features.
- In a mathematical sense, the solution to L1 minimization is ℓ_1 The ℓ_1 -norm is on the edge of the ball, so many of the weights are zero.

L2 (Ridge) এর Mathematical Property:

- The L2 penalty is convex and smooth, so the solution minimizes all weights but does not reduce them to 0.
- It works well when:
 - All the features are necessary, don't want to leave out any features.
 - The correlation between the features is high (multi-collinearity problem exists).
 - Try to prevent overfitting.
- L2 minimization solution ℓ_2 The ℓ_2 -norm is in the ball, where the weights are small but not zero.

8. Mathematical Criterion (Bias-Variance Tradeoff):

- L1 : Bias may be a little high because many weights are set to 0, reducing variance.
- L2 : Bias is reduced but variance may be a little higher.

Therefore, you need to choose according to the noise, number of features, and correlation of your data.

৫. Practical mathematical approach:

- If feature sparsity is required, i.e. you want to build a model with fewer features → choose L1 (Lasso).

- If there are correlated features or all features are needed , and you want weight shrinkage but not 0 → choose L2 (Ridge).

3. Elastic Net Regression

(Q_) When to Use:

☒ L1 Regularization (Lasso)

Use when:

- You want to remove irrelevant features automatically
- You have many features (high-dimensional data)
- Example: Text classification, gene selection in bioinformatics

☐ It performs feature selection by setting some weights to exactly 0.

(Q_) ☒ L2 Regularization (Ridge)

Use when:

- You believe all features contribute a little
- You want to prevent overfitting by shrinking large weights
- You have correlated features (multicollinearity)

❑ It reduces complexity but keeps all features.

✅ Elastic Net = L1 + L2

Use when:

- You want both sparsity and stability
- You're unsure whether L1 or L2 is better

(Q) Real-World Example:

Scenario	Regularization
House price prediction with 1000 features (many are useless)	L1
Predicting customer churn with all relevant features	L2
Text data with high sparsity (Bag-of-Words)	L1 or ElasticNet

Quick Rule

- If your model overfits → Add regularization.
- If your model underfits → Don't add regularization (or reduce it).

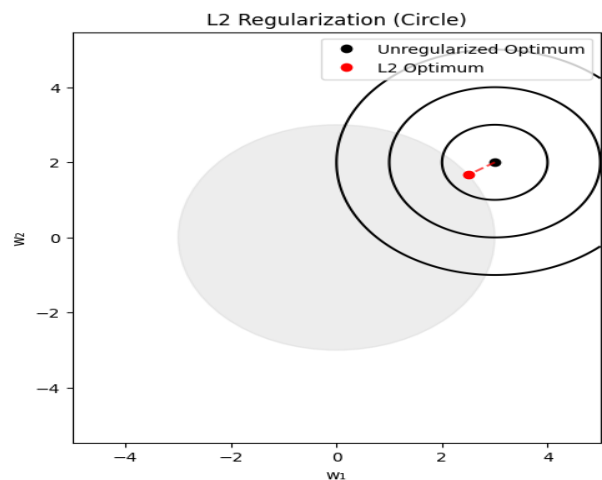
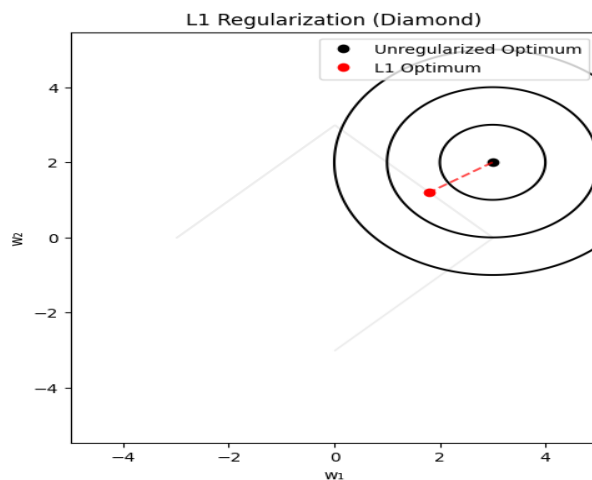
(Q) In L1 regularization (Lasso), the model understands a feature is “not important” based on how much that feature actually helps reduce prediction error after considering the regularization penalty

How it decides importance

- If a feature barely improves prediction accuracy, the improvement might not be worth the penalty cost.
- The optimization process then pushes that weight toward 0 to reduce the penalty, effectively removing the feature's influence.

(Q) In other words:

- Important feature $\rightarrow$ large enough impact on predictions $\rightarrow$ penalty cost is worth paying $\rightarrow$ weight stays non-zero.
- Unimportant feature $\rightarrow$ little to no improvement in accuracy $\rightarrow$ penalty cost is not worth it $\rightarrow$ weight becomes exactly 0.



💡 Mathematical Setup

We'll use a simple 2D quadratic loss:

$$J(w_1, w_2) = (w_1 - 3)^2 + (w_2 - 2)^2$$

That's a convex bowl centered at (3, 2) — the **unregularized optimum**.

We'll apply:

- **L1 constraint:** $|w_1| + |w_2| \leq 3$
- **L2 constraint:** $w_1^2 + w_2^2 \leq 9$

and find where the loss contour first touches each constraint.

📊 Output You'll See

The plot will show **two panels**:

🔹 Left — L1 Regularization

- The **gray diamond** is the L1 constraint ($|w_1| + |w_2| \leq 3$)
- The **black ellipses** are loss contours.
- The **black dot** (3,2) = unregularized best point.
- The **red dot** = new L1 optimum (touches the diamond).
- The **red dashed line** = how the point shrank toward zero.

👉 The ellipse touches the diamond **corner** — meaning one weight can become 0 if λ is larger.

Right — L2 Regularization

- The **gray circle** is the L2 constraint ($w_1^2 + w_2^2 \leq 9$)
- The ellipse touches the **smooth side**.
- Both weights shrink but **remain nonzero**.

Interpretation of the Picture

Feature Without Reg. With L1 With L2

w_1	3.0	1.8	2.54
w_2	2.0	1.2	1.69

- **L1 (Diamond)** → shrinks both strongly, may push one to 0 → sparse model.
- **L2 (Circle)** → shrinks both smoothly → keeps all features.